



Batch 25 — HRM Attendance + Shift Control Pack

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Tue, 16 Jun at 08:26

Batch 25 — HRM Attendance + Shift Control Pack

Meat Lovers CIMS powered by YohPal

25A.

database/migrations/054_staff_shifts.sql

```
CREATE TABLE staff_shifts (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  
  shift_name VARCHAR(255) NOT NULL,  
  start_time TIME NOT NULL,  
  end_time TIME NOT NULL,  
  
  grace_minutes INT DEFAULT 10,  
  
  shift_status ENUM(  
    'ACTIVE',  
    'INACTIVE'  
  ) DEFAULT 'ACTIVE',  
  
  notes TEXT,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

25B.

database/migrations/055_duty_rosters.sql

```
CREATE TABLE duty_rosters (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  
  staff_id BIGINT NOT NULL,  
  shift_id BIGINT NOT NULL,  
  
  duty_date DATE NOT NULL,  
  duty_department ENUM(  
    'STORE',  
    'KITCHEN',  
    'SERVICE',  
    'DISPATCH',  
    'BAR',  
    'HRM',  
    'FINANCE',  
    'ADMIN'  
  ) NOT NULL,  
  
  roster_status ENUM(  
    'SCHEDULED',  
    'COMPLETED',  
    'MISSED',  
    'CANCELLED'  
  ) DEFAULT 'SCHEDULED',  
  
  notes TEXT,  
  
  FOREIGN KEY (staff_id) REFERENCES users(id),  
  FOREIGN KEY (shift_id) REFERENCES staff_shifts(id),  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

25C.

database/migrations/056_staff_attendance.sql

```
CREATE TABLE staff_attendance (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  
  staff_id BIGINT NOT NULL,  
  duty_roster_id BIGINT,  
  
  attendance_date DATE NOT NULL,  
  
  clock_in_time DATETIME NULL,  
  clock_out_time DATETIME NULL,  
  
  attendance_status ENUM(  
    'PRESENT',  
    'LATE',  
    'ABSENT',  
    'LEFT_EARLY',  
    'INCOMPLETE'  
  ) DEFAULT 'INCOMPLETE',  
  
  lateness_minutes INT DEFAULT 0,  
  early_leave_minutes INT DEFAULT 0,  
  
  clock_in_notes TEXT,  
  clock_out_notes TEXT,  
  
  FOREIGN KEY (staff_id) REFERENCES users(id),  
  FOREIGN KEY (duty_roster_id) REFERENCES duty_rosters(id),  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

25D.

database/migrations/057_absence_reports.sql

```
CREATE TABLE absence_reports (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  
  staff_id BIGINT NOT NULL,  
  duty_roster_id BIGINT,  
  
  absence_date DATE NOT NULL,  
  
  absence_reason TEXT,  
  absence_status ENUM(  
    'REPORTED',  
    'APPROVED',  
    'REJECTED',  
    'UNEXPLAINED'  
  ) DEFAULT 'REPORTED',  
  
  reported_by BIGINT,  
  
  FOREIGN KEY (staff_id) REFERENCES users(id),  
  FOREIGN KEY (duty_roster_id) REFERENCES duty_rosters(id),  
  FOREIGN KEY (reported_by) REFERENCES users(id),  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

25E.

database/migrations/058_payroll_placeholders.sql

```
CREATE TABLE payroll_placeholders (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  
  staff_id BIGINT NOT NULL,  
  payroll_month VARCHAR(7) NOT NULL,  
  
  base_pay DECIMAL(12,2) DEFAULT 0,  
  bonus_pay DECIMAL(12,2) DEFAULT 0,  
  
  lateness_deduction DECIMAL(12,2) DEFAULT 0,  
  absence_deduction DECIMAL(12,2) DEFAULT 0,
```

```

gross_pay DECIMAL(12,2) DEFAULT 0,
net_pay DECIMAL(12,2) DEFAULT 0,

payroll_status ENUM(
    'DRAFT',
    'REVIEWED',
    'APPROVED',
    'PAID'
) DEFAULT 'DRAFT',

notes TEXT,

FOREIGN KEY (staff_id) REFERENCES users(id),

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

```

25F.

database/seeds/018_staff_shifts_seed.sql

```

INSERT INTO staff_shifts (
    shift_name,
    start_time,
    end_time,
    grace_minutes,
    shift_status,
    notes
) VALUES
('Morning Shift', '08:00:00', '16:00:00', 10, 'ACTIVE', 'Standard morning restaurant shift'),
('Evening Shift', '16:00:00', '23:00:00', 10, 'ACTIVE', 'Standard evening restaurant shift'),
('Full Day Shift', '08:00:00', '23:00:00', 15, 'ACTIVE', 'Long shift for management or special duty');

```

25G. Update

backend/routes/api.php

```

$router->get('/api/hrm/shifts', [HRMController::class, 'shifts']);
$router->post('/api/hrm/shifts', [HRMController::class, 'createShift']);

$router->get('/api/hrm/duty-rosters', [HRMController::class, 'dutyRosters']);
$router->post('/api/hrm/duty-rosters', [HRMController::class, 'createDutyRoster']);

$router->get('/api/hrm/attendance', [HRMController::class, 'attendance']);
$router->post('/api/hrm/attendance/clock-in', [HRMController::class, 'clockIn']);
$router->post('/api/hrm/attendance/clock-out', [HRMController::class, 'clockOut']);

$router->get('/api/hrm/absences', [HRMController::class, 'absences']);
$router->post('/api/hrm/absences', [HRMController::class, 'createAbsence']);

$router->get('/api/hrm/payroll-placeholders', [HRMController::class, 'payrollPlaceholders']);
$router->post('/api/hrm/payroll-placeholders', [HRMController::class, 'createPayrollPlaceholder']);

$router->get('/api/hrm/compliance-dashboard', [HRMController::class, 'complianceDashboard']);

```

25H. Update

backend/app/Controllers/HRMController.php

Add these methods inside the class:

```

public function shifts(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'HR']);

    $stmt = Database::connection()->query(
        'SELECT * FROM staff_shifts ORDER BY start_time ASC'
    );

    Response::success(['shifts' => $stmt->fetchAll()]);
}

public function createShift(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'HR']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(

```

```

        'INSERT INTO staff_shifts
        (shift_name, start_time, end_time, grace_minutes, shift_status, notes)
        VALUES (:shift_name, :start_time, :end_time, :grace_minutes, :status, :notes)'
    );

    $stmt->execute([
        'shift_name' => $data['shift_name'],
        'start_time' => $data['start_time'],
        'end_time' => $data['end_time'],
        'grace_minutes' => $data['grace_minutes'] ?? 10,
        'status' => $data['shift_status'] ?? 'ACTIVE',
        'notes' => $data['notes'] ?? null,
    ]);

    AuditLogger::log((int) $user['id'], 'HRM', 'CREATE_SHIFT', null, null, $data);

    Response::success([], 'Shift created');
}

public function dutyRosters(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'HR']);

    $stmt = Database::connection()->query(
        "SELECT
            dr.*,
            u.full_name AS staff_name,
            u.role,
            ss.shift_name,
            ss.start_time,
            ss.end_time
        FROM duty_rosters dr
        JOIN users u ON u.id = dr.staff_id
        JOIN staff_shifts ss ON ss.id = dr.shift_id
        ORDER BY dr.duty_date DESC"
    );

    Response::success(['duty_rosters' => $stmt->fetchAll()]);
}

public function createDutyRoster(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'HR']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'INSERT INTO duty_rosters
        (staff_id, shift_id, duty_date, duty_department, notes)
        VALUES (:staff_id, :shift_id, :duty_date, :department, :notes)'
    );

    $stmt->execute([
        'staff_id' => $data['staff_id'],
        'shift_id' => $data['shift_id'],
        'duty_date' => $data['duty_date'],
        'department' => $data['duty_department'],
        'notes' => $data['notes'] ?? null,
    ]);

    AuditLogger::log((int) $user['id'], 'HRM', 'CREATE_DUTY_ROSTER', (int) $data['staff_id'], null, $data);

    Response::success([], 'Duty roster created');
}

public function attendance(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'HR']);

    $stmt = Database::connection()->query(
        "SELECT
            sa.*,
            u.full_name AS staff_name,
            u.role
        FROM staff_attendance sa
        JOIN users u ON u.id = sa.staff_id
        ORDER BY sa.attendance_date DESC, sa.created_at DESC"
    );

    Response::success(['attendance' => $stmt->fetchAll()]);
}

public function clockIn(): void
{
    $user = Auth::requireAuth();
    $data = Request::body();
    $db = Database::connection();

```

```

$staffId = $data['staff_id'] ?? $user['id'];
$attendanceDate = $data['attendance_date'] ?? date('Y-m-d');
$clockIn = $data['clock_in_time'] ?? date('Y-m-d H:i:s');

$rosterStmt = $db->prepare(
    "SELECT
        dr.*,
        ss.start_time,
        ss.grace_minutes
    FROM duty_rosters dr
    JOIN staff_shifts ss ON ss.id = dr.shift_id
    WHERE dr.staff_id = :staff_id
    AND dr.duty_date = :duty_date
    LIMIT 1"
);

$rosterStmt->execute([
    'staff_id' => $staffId,
    'duty_date' => $attendanceDate,
]);

$roster = $rosterStmt->fetch();

$latenessMinutes = 0;
$status = 'PRESENT';

if ($roster) {
    $expected = strtotime($attendanceDate . ' ' . $roster['start_time']);
    $actual = strtotime($clockIn);
    $graceSeconds = ((int) $roster['grace_minutes']) * 60;

    if ($actual > ($expected + $graceSeconds)) {
        $latenessMinutes = (int) floor(($actual - $expected) / 60);
        $status = 'LATE';
    }
}

$stmt = $db->prepare(
    "INSERT INTO staff_attendance
    (staff_id, duty_roster_id, attendance_date, clock_in_time, attendance_status, lateness_minutes,
    clock_in_notes)
    VALUES (:staff_id, :duty_roster_id, :attendance_date, :clock_in_time, :status, :lateness, :notes)"
);

$stmt->execute([
    'staff_id' => $staffId,
    'duty_roster_id' => $roster['id'] ?? null,
    'attendance_date' => $attendanceDate,
    'clock_in_time' => $clockIn,
    'status' => $status,
    'lateness' => $latenessMinutes,
    'notes' => $data['clock_in_notes'] ?? null,
]);

AuditLogger::log((int) $user['id'], 'HRM', 'CLOCK_IN', (int) $staffId, null, $data);

Response::success([
    'attendance_status' => $status,
    'lateness_minutes' => $latenessMinutes,
], 'Clock-in recorded');
}

public function clockOut(): void
{
    $user = Auth::requireAuth();
    $data = Request::body();
    $db = Database::connection();

    $staffId = $data['staff_id'] ?? $user['id'];
    $attendanceDate = $data['attendance_date'] ?? date('Y-m-d');
    $clockOut = $data['clock_out_time'] ?? date('Y-m-d H:i:s');

    $attendanceStmt = $db->prepare(
        "SELECT * FROM staff_attendance
        WHERE staff_id = :staff_id
        AND attendance_date = :attendance_date
        ORDER BY created_at DESC
        LIMIT 1"
    );

    $attendanceStmt->execute([
        'staff_id' => $staffId,
        'attendance_date' => $attendanceDate,
    ]);

    $attendance = $attendanceStmt->fetch();

```

```

    if (!$attendance) {
        Response::error('No clock-in record found for this staff/date', 404);
    }

    $stmt = $db->prepare(
        'UPDATE staff_attendance
        SET clock_out_time = :clock_out_time,
            clock_out_notes = :notes,
            attendance_status = CASE
                WHEN attendance_status = "INCOMPLETE" THEN "PRESENT"
                ELSE attendance_status
            END
        WHERE id = :id'
    );

    $stmt->execute([
        'clock_out_time' => $clockOut,
        'notes' => $data['clock_out_notes'] ?? null,
        'id' => $attendance['id'],
    ]);

    AuditLogger::log((int) $user['id'], 'HRM', 'CLOCK_OUT', (int) $staffId, null, $data);

    Response::success([], 'Clock-out recorded');
}

public function absences(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'HR']);

    $stmt = Database::connection()->query(
        "SELECT
            ar.*,
            u.full_name AS staff_name,
            reporter.full_name AS reported_by_name
        FROM absence_reports ar
        JOIN users u ON u.id = ar.staff_id
        LEFT JOIN users reporter ON reporter.id = ar.reported_by
        ORDER BY ar.absence_date DESC"
    );

    Response::success(['absences' => $stmt->fetchAll()]);
}

public function createAbsence(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'HR']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'INSERT INTO absence_reports
        (staff_id, duty_roster_id, absence_date, absence_reason, absence_status, reported_by)
        VALUES (:staff_id, :duty_roster_id, :absence_date, :reason, :status, :reported_by)'
    );

    $stmt->execute([
        'staff_id' => $data['staff_id'],
        'duty_roster_id' => $data['duty_roster_id'] ?? null,
        'absence_date' => $data['absence_date'],
        'reason' => $data['absence_reason'] ?? null,
        'status' => $data['absence_status'] ?? 'REPORTED',
        'reported_by' => $user['id'],
    ]);

    AuditLogger::log((int) $user['id'], 'HRM', 'CREATE_ABSENCE_REPORT', (int) $data['staff_id'], null, $data);

    Response::success([], 'Absence report created');
}

public function payrollPlaceholders(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'HR', 'ACCOUNTANT']);

    $stmt = Database::connection()->query(
        "SELECT
            pp.*,
            u.full_name AS staff_name,
            u.role
        FROM payroll_placeholders pp
        JOIN users u ON u.id = pp.staff_id
        ORDER BY pp.payroll_month DESC"
    );

    Response::success(['payroll_placeholders' => $stmt->fetchAll()]);
}

public function createPayrollPlaceholder(): void

```

```

{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'HR', 'ACCOUNTANT']);
    $data = Request::body();

    $gross = (float) ($data['base_pay'] ?? 0) + (float) ($data['bonus_pay'] ?? 0);
    $deductions = (float) ($data['lateness_deduction'] ?? 0) + (float) ($data['absence_deduction'] ?? 0);
    $net = $gross - $deductions;

    $stmt = Database::connection()->prepare(
        'INSERT INTO payroll_placeholders
        (staff_id, payroll_month, base_pay, bonus_pay, lateness_deduction,
        absence_deduction, gross_pay, net_pay, payroll_status, notes)
        VALUES
        (:staff_id, :payroll_month, :base_pay, :bonus_pay, :lateness_deduction,
        :absence_deduction, :gross_pay, :net_pay, :status, :notes)'
    );

    $stmt->execute([
        'staff_id' => $data['staff_id'],
        'payroll_month' => $data['payroll_month'],
        'base_pay' => $data['base_pay'] ?? 0,
        'bonus_pay' => $data['bonus_pay'] ?? 0,
        'lateness_deduction' => $data['lateness_deduction'] ?? 0,
        'absence_deduction' => $data['absence_deduction'] ?? 0,
        'gross_pay' => $gross,
        'net_pay' => $net,
        'status' => $data['payroll_status'] ?? 'DRAFT',
        'notes' => $data['notes'] ?? null,
    ]);

    AuditLogger::log((int) $user['id'], 'HRM', 'CREATE_PAYROLL_PLACEHOLDER', (int) $data['staff_id'], null,
    $data);

    Response::success([], 'Payroll placeholder created');
}

public function complianceDashboard(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'HR']);

    $db = Database::connection();

    $todayAttendance = $db->query(
        "SELECT COUNT(*) AS total
        FROM staff_attendance
        WHERE attendance_date = CURDATE()"
    )->fetch();

    $lateToday = $db->query(
        "SELECT COUNT(*) AS total
        FROM staff_attendance
        WHERE attendance_date = CURDATE()
        AND attendance_status = 'LATE'"
    )->fetch();

    $absencesToday = $db->query(
        "SELECT COUNT(*) AS total
        FROM absence_reports
        WHERE absence_date = CURDATE()"
    )->fetch();

    $scheduledToday = $db->query(
        "SELECT COUNT(*) AS total
        FROM duty_rosters
        WHERE duty_date = CURDATE()"
    )->fetch();

    $payrollDrafts = $db->query(
        "SELECT COUNT(*) AS total
        FROM payroll_placeholders
        WHERE payroll_status = 'DRAFT'"
    )->fetch();

    Response::success([
        'scheduled_today' => (int) $scheduledToday['total'],
        'clocked_in_today' => (int) $todayAttendance['total'],
        'late_today' => (int) $lateToday['total'],
        'absences_today' => (int) $absencesToday['total'],
        'payroll_drafts' => (int) $payrollDrafts['total'],
    ]);
}

```

251. Update

apps/admin-web/src/features/operations/api/operationsApi.js

```
hrmShifts: async () => {
  const { data } = await apiClient.get('/hrm/shifts')
  return data.data.shifts
},

createHrmShift: async (payload) => {
  const { data } = await apiClient.post('/hrm/shifts', payload)
  return data
},

dutyRosters: async () => {
  const { data } = await apiClient.get('/hrm/duty-rosters')
  return data.data.duty_rosters
},

createDutyRoster: async (payload) => {
  const { data } = await apiClient.post('/hrm/duty-rosters', payload)
  return data
},

attendance: async () => {
  const { data } = await apiClient.get('/hrm/attendance')
  return data.data.attendance
},

clockIn: async (payload) => {
  const { data } = await apiClient.post('/hrm/attendance/clock-in', payload)
  return data
},

clockOut: async (payload) => {
  const { data } = await apiClient.post('/hrm/attendance/clock-out', payload)
  return data
},

absences: async () => {
  const { data } = await apiClient.get('/hrm/absences')
  return data.data.absences
},

createAbsence: async (payload) => {
  const { data } = await apiClient.post('/hrm/absences', payload)
  return data
},

payrollPlaceholders: async () => {
  const { data } = await apiClient.get('/hrm/payroll-placeholders')
  return data.data.payroll_placeholders
},

createPayrollPlaceholder: async (payload) => {
  const { data } = await apiClient.post('/hrm/payroll-placeholders', payload)
  return data
},

hrmComplianceDashboard: async () => {
  const { data } = await apiClient.get('/hrm/compliance-dashboard')
  return data.data
},
```

25J.

apps/admin-web/src/features/hrm/pages/HRMComplianceDashboardPage.jsx

```
import React from 'react'
import { useQuery } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function HRMComplianceDashboardPage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['hrm-compliance-dashboard'],
    queryFn: operationsApi.hrmComplianceDashboard,
  })

  return (
    <Page title="HRM Compliance Dashboard" subtitle="Attendance, lateness, absences, roster, and payroll visibility">
      {isLoading ? <LoadingBlock label="Loading HRM compliance..." /> : null}
    </Page>
  )
}
```



```

{error ? <StatusMessage error={error.message} /> : null}

{data ? (
  <div style={{ display: 'grid', gridTemplateColumns: 'repeat(5, minmax(0, 1fr))', gap: 16 }}>
    <Card title="Scheduled Today"><strong>{data.scheduled_today}</strong></Card>
    <Card title="Clocked In Today"><strong>{data.clocked_in_today}</strong></Card>
    <Card title="Late Today"><strong>{data.late_today}</strong></Card>
    <Card title="Absences Today"><strong>{data.absences_today}</strong></Card>
    <Card title="Payroll Drafts"><strong>{data.payroll_drafts}</strong></Card>
  </div>
) : null}
</Page>
)
}

```

25K.

apps/admin-web/src/features/hrm/pages/ShiftsPage.jsx

```

import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function ShiftsPage() {
  return (
    <OperationalFormPage
      title="Staff Shifts"
      subtitle="Create and manage shift definitions"
      queryKey="hrm-shifts"
      queryFn={operationsApi.hrmShifts}
      mutationFn={operationsApi.createHrmShift}
      initialForm={{
        shift_name: '',
        start_time: '',
        end_time: '',
        grace_minutes: '10',
        shift_status: 'ACTIVE',
        notes: '',
      }}
      buildPayload={(form) => ({
        ...form,
        grace_minutes: Number(form.grace_minutes || 10),
      })}
      fields={[
        { name: 'shift_name', label: 'Shift Name' },
        { name: 'start_time', label: 'Start Time', type: 'time' },
        { name: 'end_time', label: 'End Time', type: 'time' },
        { name: 'grace_minutes', label: 'Grace Minutes', type: 'number' },
        {
          name: 'shift_status',
          label: 'Status',
          type: 'select',
          options: [
            { value: 'ACTIVE', label: 'Active' },
            { value: 'INACTIVE', label: 'Inactive' },
          ],
        },
        { name: 'notes', label: 'Notes', type: 'textarea' },
      ]}
      columns={[
        { key: 'shift_name', label: 'Shift' },
        { key: 'start_time', label: 'Start' },
        { key: 'end_time', label: 'End' },
        { key: 'grace_minutes', label: 'Grace' },
        { key: 'shift_status', label: 'Status' },
      ]}
      submitLabel="Create Shift"
    />
  )
}

```

25L.

apps/admin-web/src/features/hrm/pages/DutyRosterPage.jsx

```

import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function DutyRosterPage() {
  return (
    <OperationalFormPage

```

```

title="Duty Roster"
subtitle="Schedule staff by shift, date, and department"
queryKey="duty-rosters"
queryFn={operationsApi.dutyRosters}
mutationFn={operationsApi.createDutyRoster}
initialForm={{
  staff_id: '',
  shift_id: '',
  duty_date: '',
  duty_department: 'SERVICE',
  notes: '',
}}
buildPayload=({form}) => ({
  staff_id: Number(form.staff_id),
  shift_id: Number(form.shift_id),
  duty_date: form.duty_date,
  duty_department: form.duty_department,
  notes: form.notes,
})
fields=[
  { name: 'staff_id', label: 'Staff ID', type: 'number' },
  { name: 'shift_id', label: 'Shift ID', type: 'number' },
  { name: 'duty_date', label: 'Duty Date', type: 'date' },
  {
    name: 'duty_department',
    label: 'Department',
    type: 'select',
    options: [
      { value: 'STORE', label: 'Store' },
      { value: 'KITCHEN', label: 'Kitchen' },
      { value: 'SERVICE', label: 'Service' },
      { value: 'DISPATCH', label: 'Dispatch' },
      { value: 'BAR', label: 'Bar' },
      { value: 'HRM', label: 'HRM' },
      { value: 'FINANCE', label: 'Finance' },
      { value: 'ADMIN', label: 'Admin' },
    ],
  },
  { name: 'notes', label: 'Notes', type: 'textarea' },
]
columns=[
  { key: 'staff_name', label: 'Staff' },
  { key: 'role', label: 'Role' },
  { key: 'shift_name', label: 'Shift' },
  { key: 'duty_date', label: 'Date' },
  { key: 'duty_department', label: 'Department' },
  { key: 'roster_status', label: 'Status' },
]
submitLabel="Create Duty Roster"
/>
)
}

```

25M.

apps/admin-web/src/features/hrm/pages/AttendancePage.jsx

```

import React, { useState } from 'react'
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import Button from '../../../components/ui/Button'
import Input from '../../../components/ui/Input'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function AttendancePage() {
  const qc = useQueryClient()

  const [form, setForm] = useState({
    staff_id: '',
    attendance_date: '',
    clock_in_notes: '',
    clock_out_notes: '',
  })

  const { data, isLoading, error } = useQuery({
    queryKey: ['attendance'],
    queryFn: operationsApi.attendance,
  })

  const clockInMutation = useMutation({

```

```

    mutationFn: operationsApi.clockIn,
    onSuccess: () => qc.invalidateQueries({ queryKey: ['attendance'] }),
  })

const clockOutMutation = useMutation({
  mutationFn: operationsApi.clockOut,
  onSuccess: () => qc.invalidateQueries({ queryKey: ['attendance'] }),
})

return (
  <Page title="Attendance" subtitle="Clock-in/out, lateness, and attendance status">
    <Card title="Clock In / Clock Out">
      <StatusMessage error={clockInMutation.error?.message || clockOutMutation.error?.message} />

      <div style={{ display: 'grid', gap: 10, maxWidth: 560 }}>
        <Input placeholder="Staff ID" value={form.staff_id} onChange={(e) => setForm({ ...form, staff_id:
e.target.value })} />
        <Input type="date" value={form.attendance_date} onChange={(e) => setForm({ ...form, attendance_date:
e.target.value })} />
        <Input placeholder="Clock-in notes" value={form.clock_in_notes} onChange={(e) => setForm({ ...form,
clock_in_notes: e.target.value })} />
        <Input placeholder="Clock-out notes" value={form.clock_out_notes} onChange={(e) => setForm({ ...form,
clock_out_notes: e.target.value })} />

        <div style={{ display: 'flex', gap: 10, flexWrap: 'wrap' }}>
          <Button
            onClick={() =>
              clockInMutation.mutate({
                staff_id: Number(form.staff_id),
                attendance_date: form.attendance_date,
                clock_in_notes: form.clock_in_notes,
              })
            }
          >
            Clock In
          </Button>

          <Button
            onClick={() =>
              clockOutMutation.mutate({
                staff_id: Number(form.staff_id),
                attendance_date: form.attendance_date,
                clock_out_notes: form.clock_out_notes,
              })
            }
          >
            Clock Out
          </Button>
        </div>
      </div>
    </Card>

    <Card title="Attendance Records">
      {isLoading ? <LoadingBlock label="Loading attendance..." /> : null}
      {error ? <StatusMessage error={error.message} /> : null}
      {!isLoading && !error ? (
        <DataTable
          columns={[
            { key: 'staff_name', label: 'Staff' },
            { key: 'role', label: 'Role' },
            { key: 'attendance_date', label: 'Date' },
            { key: 'clock_in_time', label: 'Clock In' },
            { key: 'clock_out_time', label: 'Clock Out' },
            { key: 'attendance_status', label: 'Status' },
            { key: 'lateness_minutes', label: 'Late Minutes' },
          ]}
          rows={data || []}
        />
      ) : null}
    </Card>
  </Page>
)
}

```

25N.

`apps/admin-web/src/features/hrm/pages/AbsenceReportsPage.jsx`

```

import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function AbsenceReportsPage() {
  return (

```

```

<OperationalFormPage
  title="Absence Reports"
  subtitle="Track staff absences and reasons"
  queryKey="absence-reports"
  queryFn={operationsApi.absences}
  mutationFn={operationsApi.createAbsence}
  initialForm={{
    staff_id: '',
    duty_roster_id: '',
    absence_date: '',
    absence_reason: '',
    absence_status: 'REPORTED',
  }}
  buildPayload={
    (form) => ({
      staff_id: Number(form.staff_id),
      duty_roster_id: form.duty_roster_id ? Number(form.duty_roster_id) : null,
      absence_date: form.absence_date,
      absence_reason: form.absence_reason,
      absence_status: form.absence_status,
    })
  }
  fields=[
    { name: 'staff_id', label: 'Staff ID', type: 'number' },
    { name: 'duty_roster_id', label: 'Duty Roster ID optional', type: 'number' },
    { name: 'absence_date', label: 'Absence Date', type: 'date' },
    { name: 'absence_reason', label: 'Reason', type: 'textarea' },
    {
      name: 'absence_status',
      label: 'Status',
      type: 'select',
      options: [
        { value: 'REPORTED', label: 'Reported' },
        { value: 'APPROVED', label: 'Approved' },
        { value: 'REJECTED', label: 'Rejected' },
        { value: 'UNEXPLAINED', label: 'Unexplained' },
      ],
    },
  ],
  columns=[
    { key: 'staff_name', label: 'Staff' },
    { key: 'absence_date', label: 'Date' },
    { key: 'absence_status', label: 'Status' },
    { key: 'absence_reason', label: 'Reason' },
    { key: 'reported_by_name', label: 'Reported By' },
  ],
  submitLabel="Record Absence"
/>
)
}

```

250.

apps/admin-web/src/features/hrm/pages/PayrollPlaceholderPage.jsx

```

import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function PayrollPlaceholderPage() {
  return (
    <OperationalFormPage
      title="Payroll Placeholder"
      subtitle="Draft payroll with lateness and absence deductions"
      queryKey="payroll-placeholders"
      queryFn={operationsApi.payrollPlaceholders}
      mutationFn={operationsApi.createPayrollPlaceholder}
      initialForm={{
        staff_id: '',
        payroll_month: '',
        base_pay: '',
        bonus_pay: '',
        lateness_deduction: '',
        absence_deduction: '',
        payroll_status: 'DRAFT',
        notes: '',
      }}
      buildPayload={
        (form) => ({
          staff_id: Number(form.staff_id),
          payroll_month: form.payroll_month,
          base_pay: Number(form.base_pay || 0),
          bonus_pay: Number(form.bonus_pay || 0),
          lateness_deduction: Number(form.lateness_deduction || 0),
          absence_deduction: Number(form.absence_deduction || 0),
          payroll_status: form.payroll_status,
          notes: form.notes,
        })
      }
    />
  )
}

```

```

    })}
    fields=[
      { name: 'staff_id', label: 'Staff ID', type: 'number' },
      { name: 'payroll_month', label: 'Payroll Month YYYY-MM' },
      { name: 'base_pay', label: 'Base Pay', type: 'number' },
      { name: 'bonus_pay', label: 'Bonus Pay', type: 'number' },
      { name: 'lateness_deduction', label: 'Lateness Deduction', type: 'number' },
      { name: 'absence_deduction', label: 'Absence Deduction', type: 'number' },
      {
        name: 'payroll_status',
        label: 'Payroll Status',
        type: 'select',
        options: [
          { value: 'DRAFT', label: 'Draft' },
          { value: 'REVIEWED', label: 'Reviewed' },
          { value: 'APPROVED', label: 'Approved' },
          { value: 'PAID', label: 'Paid' },
        ],
      },
      { name: 'notes', label: 'Notes', type: 'textarea' },
    ]
  }
  columns=[
    { key: 'staff_name', label: 'Staff' },
    { key: 'role', label: 'Role' },
    { key: 'payroll_month', label: 'Month' },
    { key: 'base_pay', label: 'Base' },
    { key: 'bonus_pay', label: 'Bonus' },
    { key: 'lateness_deduction', label: 'Late Deduction' },
    { key: 'absence_deduction', label: 'Absence Deduction' },
    { key: 'net_pay', label: 'Net Pay' },
    { key: 'payroll_status', label: 'Status' },
  ]
  submitLabel="Create Payroll Draft"
}
)
}

```

25P. Update

apps/admin-web/src/app/routes.jsx

```

import HRMComplianceDashboardPage from '../features/hrm/pages/HRMComplianceDashboardPage'
import ShiftsPage from '../features/hrm/pages/ShiftsPage'
import DutyRosterPage from '../features/hrm/pages/DutyRosterPage'
import AttendancePage from '../features/hrm/pages/AttendancePage'
import AbsenceReportsPage from '../features/hrm/pages/AbsenceReportsPage'
import PayrollPlaceholderPage from '../features/hrm/pages/PayrollPlaceholderPage'

<Route path="/hrm/compliance-dashboard" element=<HRMComplianceDashboardPage /> />
<Route path="/hrm/shifts" element=<ShiftsPage /> />
<Route path="/hrm/duty-roster" element=<DutyRosterPage /> />
<Route path="/hrm/attendance" element=<AttendancePage /> />
<Route path="/hrm/absences" element=<AbsenceReportsPage /> />
<Route path="/hrm/payroll-placeholder" element=<PayrollPlaceholderPage /> />

```

25Q. Update

apps/admin-web/src/components/common/SidebarNav.jsx

```

['HRM Compliance', '/hrm/compliance-dashboard'],
['Staff Shifts', '/hrm/shifts'],
['Duty Roster', '/hrm/duty-roster'],
['Attendance', '/hrm/attendance'],
['Absence Reports', '/hrm/absences'],
['Payroll Placeholder', '/hrm/payroll-placeholder'],

```

25R.

docs/HRM_ATTENDANCE_SHIFT_CONTROL_PACK.md

Batch 25 – HRM Attendance + Shift Control Pack

System

Meat Lovers CIMS powered by YohPal

Purpose

This batch gives Meat Lovers management control over staff shifts, attendance, lateness, absences, rosters, and payroll placeholders.

Features Added

1. Staff shifts
2. Duty roster
3. Clock-in
4. Clock-out
5. Lateness tracking
6. Absence reporting
7. Payroll placeholder
8. HRM compliance dashboard

Shift Control

Each shift has:

- shift name
- start time
- end time
- grace minutes
- status

Duty Roster

Staff can be scheduled by:

- staff member
- shift
- date
- department

Attendance

Attendance records:

- clock-in time
- clock-out time
- lateness minutes
- attendance status

Absence Tracking

Absence reports support:

- reported
- approved
- rejected
- unexplained

Payroll Placeholder

Payroll placeholder calculates:

```
```text
gross_pay = base_pay + bonus_pay
net_pay = gross_pay - lateness_deduction - absence_deduction
```

# HRM Compliance Dashboard

Dashboard shows:

- scheduled today
- clocked in today
- late today
- absences today
- payroll drafts

# Business Value

This helps Meat Lovers:

- reduce lateness
- reduce absenteeism
- enforce duty rosters
- connect attendance to payroll
- improve service team discipline

---

# Batch 25 Outcome

HRM attendance + shift control now includes:

- ✔ staff shifts
- ✔ attendance clock-in/out
- ✔ lateness tracking
- ✔ absence tracking
- ✔ duty roster
- ✔ payroll placeholder
- ✔ HRM compliance dashboard

# Next Smart Move

Build **\*\*Batch 26 – Enforcement Engine Deep Integration Pack\*\***, connecting audit logs, approvals, stock variance, cash variance, attendance violations, product pricing violations, and staff incident records into one risk-score dashboard.